AI-Powered Project Documentation & RAG System

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes all functional and non-functional requirements for the AI-driven project documentation and Retrieval Augmented Generation (RAG) system. It serves as the single source of truth for developers, testers, product owners, and AI agents involved in the project.

1.2 Scope

What the system DOES:

- Create and manage SRS documents automatically
- Analyze project requirements using AI
- Generate comprehensive test cases
- Perform semantic search using RAG technology
- Generate features from requirements documentation
- Analyze bugs and suggest fixes based on requirements

What the system does NOT do:

- Complex user interface design
- Mobile application development
- Real-time collaboration editing
- Advanced user authentication systems
- Integration with external project management tools

1.3 Definitions and Acronyms

- RAG: Retrieval Augmented Generation - AI technique that retrieves relevant documents and generates responses
- Embedding: Converting text into numerical vector representation
- Vector DB: Database specialized in storing and querying vector embeddings
- SRS: Software Requirements Specification
- API: Application Programming Interface

- Chunk: Small text segment from document splitting
- LLM: Large Language Model (e.g., GPT-4, Claude)
- Similarity Search: Finding most similar vectors in database
- Token: Basic unit of text processed by LLMs

1.4 References

- OpenAI API Documentation: https://platform.openai.com/docs
- LangChain Framework: https://js.langchain.com/docs
- MongoDB Atlas Documentation: https://www.mongodb.com/docs/atlas/
- Supabase Vector DB: https://supabase.com/docs/guides/ai
- IEEE SRS Template Standard

2. Overall Description

2.1 Product Perspective

The system is an AI-powered documentation analysis platform that:

- Serves as central hub for project requirements management
- Integrates with MongoDB for structured data storage
- Uses Supabase for vector embeddings storage
- Leverages LangChain for RAG pipeline management
- Provides web interface for project requirements management
- Acts as intelligent assistant for software development teams

2.2 Product Functions

Core Capabilities:

1. Upload SRS documents in multiple formats
2. Split documents into manageable chunks
3. Store raw text in MongoDB database
4. Generate embeddings using OpenAI API
5. Store and index embeddings in Supabase Vector DB
6. Perform semantic search using RAG technology
7. Auto-generate features from requirements
8. Generate comprehensive test cases
9. Analyze bugs and relate to requirements
10. Provide intelligent search and Q&A capabilities

2.3 User Classes and Characteristics

- Product Owner: Creates projects, uploads SRS, reviews generated features
- Developer: Searches requirements, analyzes features, implements code
- QA Engineer: Generates and executes test cases, reports bugs
- AI Agent: Automated interactions, batch processing, analysis
- Project Manager: Monovers progress, reviews requirements coverage

2.4 Operating Environment

Software Environment:

- Backend: Node.js 18+ with Express.js
- Database: MongoDB Atlas (Cloud)
- Vector Database: Supabase with pgvector
- AI Framework: LangChain.js
- AI Services: OpenAI API (GPT-4, text-embedding-ada-002)
- Frontend: React.js with TypeScript
- Containerization: Docker

Hardware Environment:

- Cloud-based deployment (AWS/Azure/GCP)
- Minimum 2GB RAM for backend services
- SSD storage for database performance

2.5 Constraints

- OpenAI API: 4096 token limit for embeddings, rate limiting
- Supabase: Vector size limits, storage capacity
- Budget: Monthly API cost constraints, cloud infrastructure costs
- Privacy: Data encryption requirements, no PII storage
- Performance: Maximum response time requirements

2.6 Assumptions and Dependencies

- Users will upload SRS documents in PDF or TXT format
- System operates in cloud environment with internet connectivity
- OpenAI API remains available and stable
- Users have basic understanding of software requirements
- Documents are in English language

- Reasonable document sizes (under 50MB)

---

3. System Features (Functional Requirements)

Feature 1: Upload SRS Document

3.1.1 Description

Allows users to upload Software Requirements Specification documents for processing and analysis.

3.1.2 Functional Requirements

- FR1.1: System shall support PDF and TXT file formats
- FR1.2: System shall extract text content from uploaded files
- FR1.3: System shall store extracted text in MongoDB database
- FR1.4: System shall perform text chunking (500-1000 word segments)
- FR1.5: System shall generate embeddings for each text chunk using OpenAI
- FR1.6: System shall store embeddings in Supabase Vector DB with metadata
- FR1.7: System shall provide upload progress indication
- FR1.8: System shall validate file size and format before processing

Feature 2: Semantic Search (RAG)

3.2.1 Description

Provides intelligent semantic search capabilities using Retrieval Augmented Generation technology.

3.2.2 Functional Requirements

- FR2.1: System shall accept natural language queries from users
- FR2.2: System shall generate embedding for search query using OpenAI
- FR2.3: System shall perform similarity search in Supabase Vector DB
- FR2.4: System shall retrieve top 5 most relevant text chunks
- FR2.5: System shall send context and query to LLM for answer generation
- FR2.6: System shall return sourced answer with references
- FR2.7: System shall handle ambiguous queries with clarification requests
- FR2.8: System shall maintain search history per user

Feature 3: Generate Features from SRS

3.3.1 Description

Automatically extracts and generates feature specifications from requirements documents.

3.3.2 Functional Requirements

- FR3.1: System shall identify requirement-related text segments
- FR3.2: System shall use LLM to generate structured feature list
- FR3.3: Each feature shall include:
    - Unique identifier (auto-generated)
    - Feature name
    - Detailed description
    - Priority level (High/Medium/Low)
    - Status (Proposed/Approved/Implemented)
    - Acceptance criteria (bullet points)
    - Related requirements references
- FR3.4: System shall allow manual review and editing of generated features
- FR3.5: System shall export features to JSON/CSV formats

Feature 4: Generate Test Cases

3.4.1 Description

Automatically creates comprehensive test cases from features and requirements.

3.4.2 Functional Requirements

- FR4.1: System shall analyze feature specifications
- FR4.2: System shall retrieve relevant context from vector database
- FR4.3: System shall generate test cases covering:
    - Positive test scenarios
    - Negative test scenarios
    - Edge cases
    - Integration test cases
- FR4.4: Each test case shall include:
    - Test case ID
    - Title and description
    - Preconditions
    - Test steps

    - Expected results
    - Priority level
    - Associated feature ID
- FR4.5: System shall support test case customization

Feature 5: Bug Analysis

3.5.1 Description

Analyzes bug reports and relates them to requirements, suggesting potential fixes.

3.5.2 Functional Requirements

- FR5.1: System shall accept bug descriptions and details
- FR5.2: System shall retrieve relevant requirements context from SRS
- FR5.3: System shall identify related requirements and features
- FR5.4: System shall suggest potential root causes
- FR5.5: System shall recommend fix approaches based on requirements
- FR5.6: System shall provide impact analysis on related features

4. External Interface Requirements

4.1 User Interfaces

Web Application Screens:

- Dashboard: Project overview and quick actions
- Upload Screen: File drag-and-drop interface with progress
- Projects Screen: List and manage all projects
- Features Screen: View, edit, and manage generated features
- Test Cases Screen: Browse and execute test cases
- Search Interface: Natural language search with results display
- Bug Analyzer: Input form for bug analysis
- Settings: Configuration and API key management

4.2 Software Interfaces

API Endpoints:

```text
```

```
POST   /api/projects              - Create new project
POST   /api/upload                - Upload SRS document
GET    /api/projects/:id          - Get project details
POST   /api/embeddings/generate   - Generate embeddings for document
POST   /api/search                - Semantic search
POST   /api/features/generate     - Generate features from SRS
GET    /api/features/:projectId   - Get project features
POST   /api/tests/generate        - Generate test cases
POST   /api/bugs/analyze          - Analyze bug report
DELETE /api/projects/:id          - Delete project
```

External Services:

- OpenAI API: For embeddings and LLM completions
- MongoDB Atlas: Primary database service
- Supabase: Vector database for embeddings
- Cloud Storage: File storage for uploaded documents

4.3 Hardware Interfaces

- Standard cloud server infrastructure
- No special hardware requirements
- Internet connectivity for API calls
- Sufficient storage for document and vector databases

5. Non-Functional Requirements

5.1 Performance Requirements

- Search Response Time: < 2 seconds for semantic search queries
- Document Processing: < 10 seconds for average documents (under 50 pages)
- API Response Time: < 500ms for non-AI endpoints
- Concurrent Users: Support for 50+ simultaneous users
- Upload Speed: Process 10MB file within 30 seconds
- Database Queries: < 100ms for standard queries

5.2 Security Requirements

- Data Encryption: All data encrypted in transit (TLS 1.2+) and at rest
- API Keys: Secure storage and rotation of OpenAI API keys
- Authentication: JWT-based user authentication
- Authorization: Role-based access control
- Data Privacy: No storage of personal identifiable information
- Audit Logging: All operations logged for security monitoring

5.3 Usability Requirements

- Learning Curve: Users should be productive within 30 minutes
- Interface: Intuitive, clean, and responsive design
- Accessibility: WCAG 2.1 AA compliance
- Error Handling: Clear, actionable error messages
- Documentation: Comprehensive user guides and tooltips

5.4 Reliability Requirements

- Availability: 99.5% uptime excluding scheduled maintenance
- Error Recovery: Automatic retry mechanism for failed API calls
- Data Integrity: Backup and recovery procedures
- Failure Handling: Graceful degradation when external services fail
- Monitoring: Real-time health checks and alerts

5.5 Scalability Requirements

- Horizontal Scaling: Support for multiple backend instances
- Database Scaling: MongoDB Atlas auto-scaling configuration
- Vector DB: Supabase scalable infrastructure
- Load Handling: Support for increasing document volumes
- Cost Management: Efficient use of AI API resources

---

6. Database Requirements

6.1 MongoDB Schema

```javascript
// Projects collection
```

```
{
  projectId: ObjectId,
  name: String,
  description: String,
  createdAt: Date,
  updatedAt: Date,
  createdBy: String,
  status: String, // 'active', 'archived'
  settings: Object
}

// Documents collection
{
  documentId: ObjectId,
  projectId: ObjectId,
  filename: String,
  originalText: String,
  fileSize: Number,
  mimeType: String,
  uploadDate: Date,
  chunks: [{
    chunkId: String,
    text: String,
    pageNumber: Number,
    chunkIndex: Number,
    tokenCount: Number
  }],
  processingStatus: String // 'pending', 'processing', 'completed', 'failed'
}

// Features collection
{
  featureId: ObjectId,
  projectId: ObjectId,
  name: String,
  description: String,
  priority: String, // 'HIGH', 'MEDIUM', 'LOW'
  status: String, // 'PROPOSED', 'APPROVED', 'IMPLEMENTED', 'TESTED'
  acceptanceCriteria: [String],
  relatedChunks: [String],
  generatedAt: Date,
  lastModified: Date,
  createdBy: String
}

// Test Cases collection
```

```
{
  testCaseId: ObjectId,
  projectId: ObjectId,
  featureId: ObjectId,
  title: String,
  description: String,
  steps: [String],
  expectedResult: String,
  priority: String, // 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'
  status: String, // 'DRAFT', 'READY', 'PASSED', 'FAILED'
  testData: String,
  preconditions: [String],
  generatedAt: Date

}
```

6.2 Supabase Vector Database Schema

```sql
-- Embeddings table
CREATE TABLE embeddings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id TEXT NOT NULL,
    content TEXT NOT NULL,
    metadata JSONB NOT NULL,
    embedding VECTOR(1536),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    chunk_id TEXT,
    document_id TEXT
);

-- Create index for similarity search

CREATE INDEX ON embeddings USING ivfflat (embedding vector_cosine_ops);
```

---

7. System Architecture

7.1 High-Level Architecture Diagram

```text
```

```
┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│   Web Client     │     │   API Gateway    │     │   Auth Service   │
│   (React.js)     │◄──►│   (Express.js)   │◄──►│   (JWT)          │
└──────────────────┘     └──────────────────┘     └──────────────────┘
                                  │
                                  │
┌──────────────────────────────────────────────────────────────────┐
│                        Application Layer                          │
└──────────────────────────────────────────────────────────────────┘
                                  │
            ┌─────────────────────┼─────────────────────┐
            │                     │                     │
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│   Document       │   │   RAG Service    │   │   Feature        │
│   Processing     │   │   (LangChain)    │   │   Generator      │
│   Service        │   │                  │   │   Service        │
└──────────────────┘   └──────────────────┘   └──────────────────┘
            │                     │                     │
            └─────────────────────┼─────────────────────┘
                                  │
┌──────────────────────────────────────────────────────────────────┐
│                        Data Layer                                 │
└──────────────────────────────────────────────────────────────────┘
                                  │
            ┌─────────────────────┼─────────────────────┐
            │                     │                     │
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│   MongoDB        │   │   Supabase       │   │   OpenAI API     │
│   (Documents)    │   │   (Vectors)      │   │   (AI Services)  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

7.2 Data Flow Description

1. Document Upload Flow:
   User → File Upload → Text Extraction → Chunking → Embedding Generation → Vector Storage
2. Search Flow:
   User Query → Query Embedding → Similarity Search → Context Retrieval → LLM Processing → Answer Generation
3. Feature Generation Flow:
   SRS Document → Requirement Extraction → LLM Analysis → Feature Structuring → Database Storage

8. Appendices

8.1 Example Feature Output

```json
{
  "featureId": "FEAT-001",
  "name": "User Authentication System",
  "description": "Implement secure user authentication using email and
password with JWT tokens",
  "priority": "HIGH",
  "status": "PROPOSED",
  "acceptanceCriteria": [
    "Users can register with valid email and password",
    "Password must be at least 8 characters with special characters",
    "Successful login returns JWT token",
    "Failed login attempts are logged and limited",
    "Tokens expire after 24 hours requiring re-authentication"
  ],
  "relatedRequirements": ["REQ-001", "REQ-005", "REQ-012"],
  "estimatedComplexity": "MEDIUM"

}
```

8.2 Example Test Case Output

```json
{
  "testCaseId": "TC-001",
  "title": "Verify successful user login with valid credentials",
  "description": "Test that users can login successfully with correct email
and password",
  "priority": "HIGH",
  "status": "DRAFT",
  "preconditions": [
    "User account exists in system",
    "User is not currently logged in"
  ],
  "testSteps": [
```

```
    "Navigate to login page",
    "Enter valid email address",
    "Enter correct password",
    "Click login button"
  ],
  "expectedResult": "User is redirected to dashboard and JWT token is
stored",
  "testData": {
    "email": "test@example.com",
    "password": "SecurePass123!"
  }
}
```

8.3 Example Bug Analysis Output

```json
{
  "bugId": "BUG-001",
  "description": "Login fails with correct credentials after password
change",
  "analysis": {
    "relatedRequirements": ["REQ-005", "REQ-008"],
    "rootCause": "Password hash not updating in session cache",
    "suggestedFix": "Clear user session cache on password change",
    "impactedFeatures": ["FEAT-001", "FEAT-003"],
    "priority": "HIGH",
    "estimatedEffort": "2 hours"
  }
}
```

8.4 API Response Examples

Success Response:

```json
{
  "success": true,
  "data": {
    "projectId": "PROJ-123456",
    "name": "E-commerce Platform",
```

```
    "featuresGenerated": 15,
    "processingTime": "8.2 seconds"
  },
  "message": "Project processed successfully"
}
```

Error Response:

```
{
  "success": false,
  "error": {
    "code": "API_RATE_LIMIT",
    "message": "OpenAI API rate limit exceeded",
    "retryAfter": "30 seconds"
  },
  "suggestion": "Please wait 30 seconds before retrying"
}
```